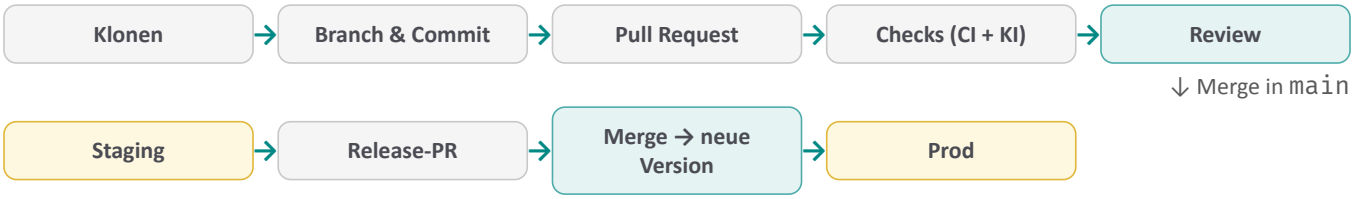
Git & GitHub bei MeKo — die Take-aways

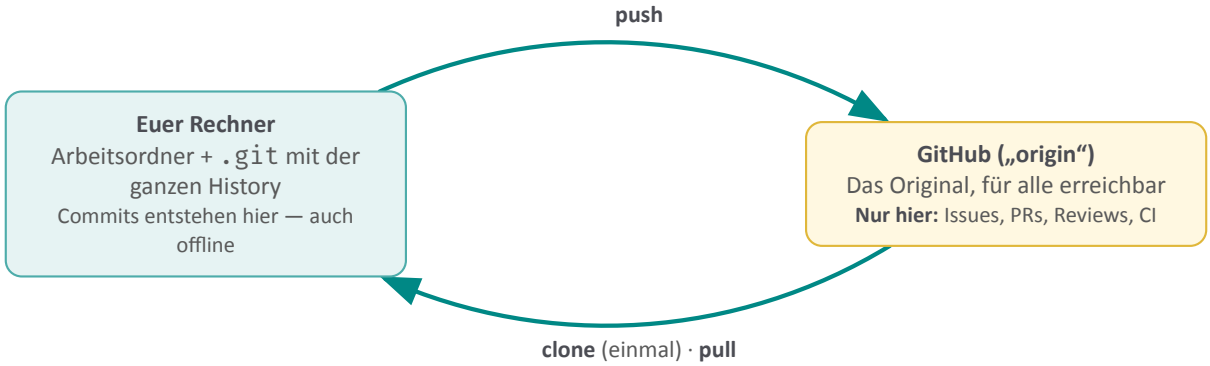
Repo `MeKo-Christian/workshop-teamwand` · Seite `meko-christian.github.io/workshop-teamwand` (nach jedem Merge, Version im Footer)

Der Weg einer Änderung



Niemand ändert `main` direkt — nicht ihr, nicht der Agent, nicht die Admins. Jede Änderung nimmt diesen Weg, auch die kleinste. Der Agent tippt die Befehle; ihr sagt ihm, was er tun soll, und **prüft das Ergebnis**.

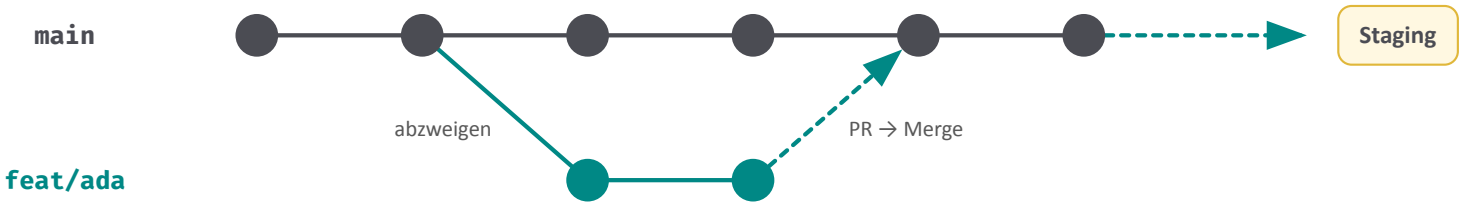
Zwei Kopien: euer Rechner und GitHub



- Nichts gleicht sich von selbst ab: **ihr ändert** → GitHub sieht es erst nach dem **push**. **Andere ändern** → ihr seht es erst nach dem **pull**.
- **Push abgelehnt?** GitHub hat auf dem Branch Commits, die ihr nicht habt → erst **pull**.
- **Ordner gelöscht?** GitHub bleibt unverändert, neu klonen. Verloren ist nur, was nie gepusht wurde.
- Ein Merge in `main` blockiert euren Push nicht; `main` holt ihr eigens in euren Branch.

Faustregel: vor der Arbeit pull, nach dem Commit push.

`main` und Branch



- `main` = der **abgestimmte Stand**, den ihr beim Klonen bekommt. Der Name ist frei wählbar (früher meist `master`).
- **Unsere Entscheidung:** jeder Merge in `main` geht automatisch auf Staging.
- Neue Arbeit startet immer vom neuesten `main` → erst pull.
- **Ein Issue = ein Branch = ein PR:** klein, gut zu reviewen.
- Name: Typ + Kurzbeschreibung, z. B. `feat/ada`.
- Ein Branch kostet nichts (nur ein Name), entsteht lokal, wird nach dem Merge gelöscht. Schiefgegangen? Wegwerfen.

Der PR-Titel wird zum Changelog

Beim Merge wird der PR-Titel **wörtlich** der Commit auf `main` und landet in den Release-Notes. Darum hat er eine feste Form, und die CI prüft sie:

`typ: was sich ändert`

Gut

```
feat: add anna to the
team wall
fix: keep long mottos
inside the card
```

Fällt durch die CI

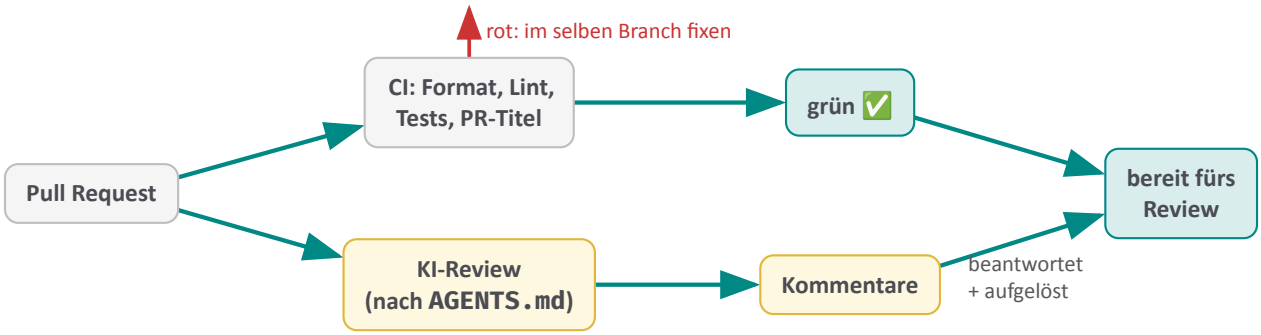
```
Update team.js
fix stuff
```

Formal ok, sagt nichts

```
feat: changes
```

`feat`	neue Funktion	`docs`	nur Dokumentation
`fix`	Fehler behoben	`refactor`	Umbau ohne neues Verhalten
`test`	nur Tests	`chore`	Sonstiges (Abhängigkeiten, Aufräumen)

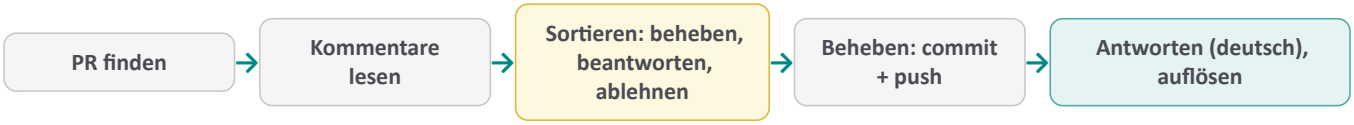
Checks und KI-Review



- Beides läuft **automatisch** bei jedem PR. **Rot = kein Merge.**
- Rot wird **im selben Branch** repariert, nie werden Regeln abgeschaltet. Rot ist gut: der Fehler ist gefunden, bevor ihn jemand sieht.
- KI-Kommentare **bewerten**, nicht blind umsetzen.
- Jeder Thread wird beantwortet und aufgelöst, sonst lässt sich nicht mergen. Der KI-Review ist beratend, die Freigabe gibt ein Mensch.

Skill `/pr-fix`

Ein **Skill** ist eine Anleitung als Markdown-Datei, die der Agent Schritt für Schritt abarbeitet. `/pr-fix` erledigt die offenen Review-Kommentare eures PRs:



Eingebaute Bremsen

- Kein offener PR → tut nichts.
- Fragen werden beantwortet, nicht aufgelöst.
- Erst push, dann antworten — nie umgekehrt.
- Unklarer Kommentar → fragt euch statt zu raten.

Eure Rolle

- Die Sortierung prüfen, **bevor** er ändert.
- Ablehnen ist erlaubt — mit Begründung.
- Die Antworten stehen unter **eurem Namen**: lesen!

Review: Vier Augen

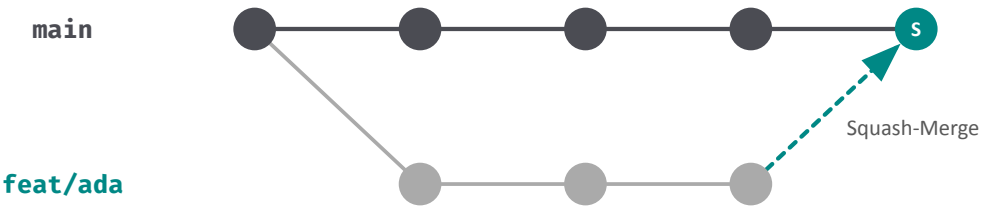
Ihr als Reviewer:in

- Passt der PR zum Issue — und **nur** dazu?
- Selbst ansehen, nicht nur dem Agenten glauben.
- Ist der Titel ein guter Changelog-Eintrag?
- Konkret kommentieren: **was** und **warum**. Änderungen anfordern ist nicht unhöflich.

Der KI-Reviewer

- Findet Muster, schnell und gründlich.
- Kennt euer Ziel nicht und kann irren.
- **Gibt nichts frei:** Das macht immer ein Mensch, nie der Autor selbst. Approve heißt: ich stehe mit dafür ein.

Merge und Konflikt



= ein Commit mit PR-Titel

- **Squash-Merge:** der ganze PR wird **ein** Commit S auf `main`, mit dem PR-Titel als Nachricht. `main` bleibt eine gerade Linie, egal wie es im Branch aussah.
- Jeder Merge ändert `main`: mergt **nacheinander**, die anderen ziehen nach. Danach lokal **pull**, sonst kennt euer Rechner den neuen Stand nicht.

Konflikt: zwei PRs ändern **dieselbe Stelle**. Wer zuerst merged, gewinnt, der andere zieht nach.

```
<<<<<<< main
  { login: "anna", … },
=======
  { login: "ben", … },
>>>>>>> mein-branch
```

Was richtig ist, wisst ihr

Git kann nicht entscheiden, welche Zeile gilt. In der Teamwand: **beide** behalten, alphabetisch sortiert.

„Löse den Konflikt so, dass beide Einträge erhalten bleiben, und erklär mir, was du gemacht hast."

Release: Versionen aus PR-Titeln



release-please hält nach jedem Merge **einen** Release-PR offen:

- Changelog aus euren **PR-Titeln**
- neue Versionsnummer
- Merge des Release-PRs → **Prod**

Prod bekommt genau den Stand, der schon auf Staging lief.

chore(main): release 0.2.0

```
## 0.2.0

### Features
* add anna to the team wall
* add ben to the team wall
* add carla to the team wall
```

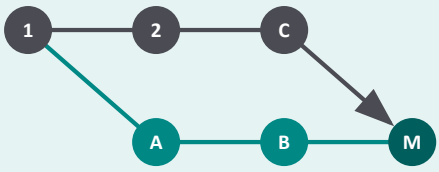
Und wenn Prod kaputt ist?

Kein Hotfix per Hand. Die Änderung per **Revert-PR** zurücknehmen und neu releasen — gleicher Weg, auch unter Zeitdruck.

Merge oder Rebase? Euren Branch auf `main` nachziehen

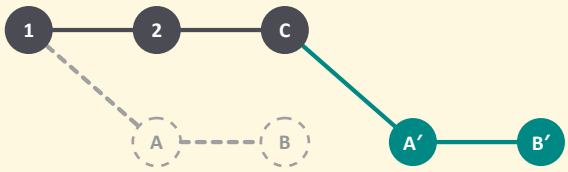
Euer Branch ist hinter `main`, weil inzwischen C gemerged wurde. Zwei Wege:

Merge — der sichere Standard



Ein Merge-Commit M verbindet beide Linien; A und B bleiben unverändert. Das macht auch „Update branch" auf GitHub.

Rebase — neu aufsetzen



Eure Commits werden auf das neue `main` neu abgespielt → **neue** Commits A′, B′. Danach braucht der push `--force-with-lease`.

Goldene Regel

Nie Commits rebasen, die andere schon haben — sonst gibt es doppelte Commits. `main` ist gegen force push gesperrt. Der Code ist nach beiden Wegen derselbe, nur die History nicht.

Prompts, Etappe für Etappe

Ihr sagt eurem Agenten, **was** er tun soll, und prüft danach selbst:

Etappe	Sagt eurem Agenten	Selbst prüfen
1 · Klonen	**„Klone MeKo-Christian/workshop-teamwand in meinen Projektordner.“ „Welche Issues sind mir zugewiesen?“**	`index.html` im Browser öffnen
2 · Branch & Commit	**„Leg einen Branch nach AGENTS.md an, erstelle meine Karte (Name, Emoji, Motto) und committe. Noch nicht hochladen.“**	Karte lokal sichtbar? Branch auf GitHub? (noch nicht)
3 · Push & PR	**„Lade meinen Branch hoch und öffne einen PR mit Conventional-Commit-Titel. Gib mir den Link.“**	Titel verständlich? Nur eure Zeile geändert?
4 · Checks	**„Sind die Checks grün? Falls rot: Lies das Log und behebe den Fehler, ohne Regeln abzuschalten.“ „Fass das KI-Review zusammen. Was ist berechtigt?“**	Kommentare beantworten und auflösen
5 · Review	**„Fordere ein Review von der Person an, die in meinem Issue steht.“ „Hol mir diesen PR und fass zusammen: Passt er zum Issue und zu AGENTS.md?“**	Fremden PR ansehen, **selbst** freigeben
6 · Merge & Staging	**„Ist mein PR freigegeben und grün? Dann merge ihn.“ „Hol den neuesten Stand von main und lösch meinen alten Branch lokal.“**	Staging zeigt eure Karte
7 · Release & Prod	**„Zeig mir den offenen Release-PR und fass den Changelog zusammen.“ „Ist die Release-Pipeline durchgelaufen?“**	Gemeinsam freigeben, Prod und Version prüfen

Wenn es hakt: „Mein PR hat einen Konflikt mit main …“ · „Bring meinen Branch per Merge auf den Stand von main.“ · „Erklär mir, was du gerade gemacht hast und warum.“

Wer macht was?

Der Agent tippt

- Befehle ausführen: klonen, Branch, Commit, push
- PR anlegen, Logs lesen, Fehler beheben
- Kommentare und Changelog zusammenfassen
- Konflikte vorschlagen aufzulösen

Ihr entscheidet und prüft

- Was geändert wird und ob es zum Issue passt
- Titel, Text und Diff vor dem Hochladen
- Welche KI-Kommentare berechtigt sind
- **Die Freigabe im Browser** — nie der Agent

Eure Regeln auf einen Blick

- Niemand ändert `main` direkt.
- Ein Issue, ein Branch, ein kleiner PR.
- Vor der Arbeit pull, nach dem Commit push.
- PR-Titel: `typ: was sich ändert`.
- Rote CI wird repariert, nicht abgeschaltet.
- Jeder KI-Kommentar wird bewertet und beantwortet.
- Das Review ist ein Mensch, nie der Autor.
- Prod nur über den Release-PR; Rollback per Revert-PR.

Der Agent tippt. Verantwortlich seid ihr — und das Review ist eure Unterschrift.

Weiterüben: Issues mit Label `bonus` (kleine Features mit Test, gleicher Weg) · Vorlage für euer eigenes Projekt: `MeKo-Tech/go-react-template`